

第二十七届华南大学生物理实验设计大赛

命题类作品

研究报告

赛道：大学物理理论课辅助教学微视频

课题：光栅衍射

2026 年 8 月 28 日

摘要

本项目通过融合理论分析、实验验证与数值仿真，系统研究了单缝及多种形状孔径的夫琅和费衍射现象。理论层面，基于惠更斯-菲涅耳原理，结合菲涅耳半波带法与衍射积分法，对衍射光强分布进行定性解释与数学推导，建立单缝衍射的理论模型，并拓展至方孔、圆孔及正十二边形孔等不同孔径，分析其远场衍射图样的特征与形成机制。实验采用激光光源构建光学系统，成功观测到单缝的明暗条纹、方孔的网格状光斑以及正十二边形孔产生的星芒结构，实验结果与理论预测高度一致。为提升教学效果，利用 Mathematica 进行数值仿真，结合 Manim 动画技术制作教学微视频，形象展示衍射图样的形成过程，并融入摄影中常见的“星芒”图案等实际应用案例，增强理论与实践的联系。项目创新性地将多孔径对比实验、数值仿真与动画演示相结合，构建了集科学性、直观性与趣味性于一体的光学教学新模式，有效提升了夫琅和费衍射的教学效果与传播力。

关键词: 夫琅和费衍射;微视频;惠更斯-菲涅耳原理;傅里叶变换

目录

- 1 项目概述3
 - 1.1 项目背景3
 - 1.2 项目简介3
- 2 基本理论3
 - 2.1 惠更斯 - 菲涅耳原理 3
 - 2.2 夫琅禾费衍射基础4
 - 2.3 单缝夫琅禾费衍射5
 - 2.4 多缝干涉原理6
 - 2.5 光栅衍射与缺级现象 7
 - 2.6 光栅半角宽度与色分辨本领 7
- 3 实验与仿真设计9
 - 3.1 光栅衍射仿真设计9
 - 3.2 衍射现象验证实验 13
- 4 视频设计与呈现 15
 - 4.1 视频结构设计 15
 - 4.2 制作工具 16
 - 4.3 动画实现 17
 - 4.4 多媒体素材整合20
- 5 创新点与总结21
 - 5.1 创新点21
 - 5.2 总结22
- 6 参考资料22
- 附件 1：组员分工 23
- 附件 2：全部代码24

1 项目概述

1.1 项目背景

光栅衍射是波动光学的核心知识点，也是现代精密光学工程的物理基础。从芯片制造核心的 EUV 光刻机，到大视场光谱巡天的郭守敬望远镜，再到可控核聚变“人造太阳”的光谱诊断系统，光栅器件的性能直接决定了高端制造与前沿科研的精度上限。

在传统大学物理教学与大众科普中，光栅衍射长期存在“原理抽象、理解困难”的痛点：一方面，单缝衍射、多缝干涉、缺级现象等知识点涉及复杂的矢量叠加与数学推导，仅靠静态公式与示意图难以建立直观的物理图像；另一方面，多数科普内容仅停留在理论讲解层面，未能将基础原理与大国重器的工程应用建立关联，难以让受众感知物理知识的现实价值。

伴随短视频与新媒体科普的快速发展，受众对科学内容的可视化、故事化、互动化需求持续提升。以动画微视频为载体，将抽象的光学原理转化为直观的动态演示，同时结合前沿科技应用场景，成为提升物理科普效果与教学效率的重要方向。本项目正是基于这一背景，围绕光栅衍射主题开发系统化的科普微视频作品。

1.2 项目简介

本项目以“光栅衍射”为核心主题，制作时长约 3 分钟的科普微视频，同时配套开发可交互的光栅衍射仿真体验工具。

项目以波动光学理论为学术基础，遵循“现象引入 — 原理拆解 — 特性分析 — 应用升华”的认知逻辑，系统讲解惠更斯 - 菲涅耳原理、单缝夫琅禾费衍射、多缝干涉、光栅衍射缺级、半角宽度与色分辨本领等核心知识点。视频融合三维光场动画、二维光路推导、实拍素材与前沿工程案例，将抽象的数学物理过程转化为可视化的动态画面，降低理解门槛。

项目核心产出包括：1 支完整的光栅衍射科普微视频、全套原理动画素材、1 个可参数调节的光栅衍射互动网页。作品既可以作为大学物理实验的辅助教学资源，也可以作为面向大众的精密光学科普内容，帮助受众理解光栅技术的科学价值，感知我国在高端光学领域的技术突破。

2 基本理论

2.1 惠更斯 - 菲涅耳原理

惠更斯 - 菲涅耳原理是研究衍射现象的理论基础，其核心思想可以表述为：**波前上的每一个面元都可以看作是发射子波的波源，空间中任意一点的光振动，是所有这些子**

波在该点的相干叠加。

惠更斯最早提出‘子波’的概念，能定性说明波绕过障碍物的传播方向（即衍射现象），但无法解释衍射图样的光强分布。菲涅耳在此基础上补充了‘子波相干叠加’的假设，认为各子波在空间相遇时产生干涉效应，空间某点的光振动由所有子波在该点的振幅与相位相干叠加决定。其数学表达式为：

$$U(P) = C \int_{\sigma} U(Q) K(\theta) \frac{e^{ikr}}{r} dS$$

其中：

$U(P)$ 为空间场点 P 的复振幅；

$U(Q)$ 为波前上某点 Q （次级波源）处的波扰， C 为比例常数；

$K(\theta)$ 为倾斜因子，描述子波振幅随衍射角 θ 的变化规律；

dS 为波前 S 上的面元； r 为面元到场点 P 的距离； k 为波数， $k = \frac{2\pi}{\lambda}$ ；

该原理表明，一切波传播的现象可以用无穷多子波的叠加来解释，于是光栅衍射现象可以通过子波相干叠加的思想进行分析与计算。

2.2 夫琅禾费衍射基础

根据光源、衍射屏、接收屏三者的相对位置，衍射通常分为两大类：菲涅耳衍射（近场衍射）与夫琅禾费衍射（远场衍射）。

菲涅耳衍射：光源和接收屏（或其中之一）距离衍射屏为有限远，入射波与衍射波均为球面波，计算相对复杂。

夫琅禾费衍射：光源和接收屏都距离衍射屏无穷远，入射光与衍射光均为平行光，与菲涅耳衍射的远场近似。在实际实验中，通常借助两个透镜实现：第一个透镜将点光源发出的光转化为平行光入射到衍射屏，第二个透镜将衍射后的平行光会聚到焦平面上进行观察。

夫琅禾费衍射的近似条件为：衍射屏到场点的距离 z 满足 $z \gg \frac{a^2}{\lambda}$ ，其中 a 为衍射孔的线度， λ 为入射光波长。在该近似下，衍射积分的计算可以大幅简化，同时衍射图样的相对强度分布不随传播距离变化，整体几何尺寸与距离 z 成正比。具有稳定的分布规律。

夫琅禾费衍射是光栅光谱仪的核心工作模式，也是光刻机等光学仪器投影成像的基础理论框架之一，本项目将以此为重点讲解衍射类型。

2.3 单缝夫琅禾费衍射

单缝夫琅禾费衍射是研究光栅衍射的基础。设狭缝宽度为 a ，波长为 λ 的平行光垂直

入射，衍射角为 θ 的平行光经透镜会聚到焦平面上。

物理过程

将狭缝处的波前沿缝宽方向分割为无穷多个等宽的子波源，每个子波源向各个方向发射子波。衍射角为 θ 的一组平行光，从波前不同位置出发时存在光程差，到达接收屏同一点时具有不同的相位，通过相干叠加形成最终的光强分布。

光强公式推导

通过衍射积分计算可得，单缝夫琅禾费衍射的光强分布为：

$$I = I_0 \left(\frac{\sin \alpha}{\alpha} \right)^2$$

其中， I_0 为中央主极大的光强； α 为单缝衍射的相位参数，物理意义是狭缝边缘与中心的子波在衍射角 θ 方向的相位差的一半，表达式为：

$$\alpha = \frac{\pi a \sin \theta}{\lambda}$$

衍射图样特征

1. **主极大位置**：当 $\alpha = 0$ （即 $\theta = 0$ ）时， $\frac{\sin \alpha}{\alpha} = 1$ ，光强取得最大值 I_0 ，对应中央亮纹，能量主要集中在中央亮纹内。
2. **极小值位置**：当 $\alpha = \pm k\pi$ ($k = 1, 2, 3 \dots$)，即 $a \sin \theta = \pm k\lambda$ 时，光强为 0，对应暗纹位置。
3. **次级大**：相邻两个暗纹之间存在一个次极大，位置满足 $\tan \alpha = \alpha$ ，其光强随级次增大迅速衰减，第一级次极大光强仅为中央主极大的约 4.7%。
4. **条纹宽度**：中央亮纹的角宽度为两侧第一级暗纹的夹角， $\Delta\theta_0 = \frac{2\lambda}{a}$ ；其他各级亮纹的角宽度为中央亮纹的一半， $\Delta\theta = \frac{\lambda}{a}$ 。缝宽越小，衍射效应越显著，条纹越宽。

2.4 多缝干涉原理

光栅由大量等宽、等间距的平行狭缝组成，设缝数为 N ，相邻两缝的中心间距为 d （称为光栅常数）。分析多缝干涉时，先忽略单缝衍射的调制作用，将每条狭缝近似为一个点波源。

相位差与光强分布

相邻两缝发出的衍射角为 θ 的平行光，到达接收屏同一点的光程差为 $d\sin\theta$ ，对应的相位差为：

$$\delta = \frac{2\pi}{\lambda} d\sin\theta$$

令 $\beta = \frac{\delta}{2} = \frac{\pi d\sin\theta}{\lambda}$ ，即相邻两束光相位差的一半。

将 N 个缝的光振动用矢量表示，通过矢量叠加法或复振幅求和，可推导出多缝干涉的光强分布：

$$I = I_1 \left(\frac{\sin N\beta}{\sin \beta} \right)^2$$

其中， I_1 为单条缝在该方向的光强。

干涉主极大与光栅方程

当相邻两缝的光程差为波长的整数倍，即 $d\sin\theta = k\lambda$ ($k = 0, \pm 1, \pm 2, \dots$) 时， $\beta = k\pi$ ，此时 $\frac{\sin N\beta}{\sin \beta} = N$ ，所有缝的光振动同相位，总振幅为单缝的 N 倍，总光强为单缝的 N^2 倍，光强达到峰值，称为**干涉主极大**。

公式 $d\sin\theta = k\lambda$ 即为**光栅方程**，是光栅最核心的规律。它表明不同波长的光对应不同的衍射角，这是光栅能够分光的物理基础。

干涉图样特征

主极大条纹细锐明亮，光强远高于次极大；

两个相邻主极大之间，分布着 $N - 1$ 条暗纹和 $N - 2$ 个次极大；

缝数 N 越大，主极大条纹越细越亮，背景越暗，分光效果越好。

2.5 光栅衍射与缺级现象

实际的光栅衍射是单缝衍射效应与多缝干涉效应的共同结果：单缝衍射对各个方向的光强进行调制，多缝干涉决定主极大的位置与精细结构。

光栅衍射总光强

将单缝衍射因子与多缝干涉因子相乘，得到光栅衍射的总光强分布：

$$I = I_0 \left(\frac{\sin \alpha}{\alpha} \right)^2 \left(\frac{\sin N\beta}{\sin \beta} \right)^2$$

其中, $\alpha = \frac{\pi a \sin \theta}{\lambda}$, $\beta = \frac{\pi d \sin \theta}{\lambda}$ 。

缺级现象的成因

如果某一级干涉主极大的位置, 恰好与单缝衍射的极小值位置重合, 那么该级主极大的光强会被调制为 0, 对应的亮纹完全消失, 这种现象称为**缺级**。

缺级满足的条件为:

$$\begin{cases} d \sin \theta = k\lambda \\ a \sin \theta = k'\lambda \end{cases}$$

联立两式可推导出缺级级次公式:

$$k = \frac{d}{a} k' (k' = \pm 1, \pm 2, \pm 3 \cdots)$$

当光栅常数与单缝缝宽的比值为整数时, 对应整数倍衍射级次会全部消失。缺级现象直观证明了光栅衍射是「单缝衍射包络调制多缝干涉条纹」的复合光学现象, 是光栅衍射区别于单纯多缝干涉的核心特征。

2.6 光栅半角宽度与色分辨本领

2.6.1 半角宽度

光栅衍射主极大亮纹并非理想几何线条, 具备一定角宽度, 工程与光学教学中统一使用**半角宽度**定量描述亮纹精细度: 定义为衍射主极大中心位置到相邻第一级暗纹的角距离。

根据光栅暗纹条件与光栅方程微分推导, 第 k 级主极大半角宽度公式为:

$$\Delta\theta = \frac{\lambda}{Nd \cos \theta_k}$$

式中: N 为光栅总刻缝数, d 为光栅常数, θ_k 为第 k 级衍射角。由公式可得核心规律: 光栅总缝数 N 越大, 半角宽度越小, 衍射亮纹越锐利、背景暗区越干净, 分光精度越高。这也是高精度光学光栅均采用高密度、多刻线设计的理论依据。

2.6.2 色分辨本领

光栅的核心工程价值为分光能力, 能够区分波长极接近的复合光谱, 该性能指标定义为**色分辨本领**。行业与物理教学统一采用**瑞利判据**作为分辨临界标准: 一条谱线的主极大峰值恰好落在相邻谱线的第一级暗纹位置时, 认定两条谱线刚好可分辨。

设可分辨的最小波长差为 $\Delta\lambda$ ，中心波长为 λ ，联立光栅方程与半角宽度公式，推导出光栅色分辨本领公式：

$$R = \frac{\lambda}{\Delta\lambda} = kN$$

该公式表明：光栅分辨本领仅由**衍射级次 k** 和**总刻缝数 N** 决定，与光栅常数无关。在实际工程应用中，提升光谱分辨精度的核心手段为：选用更高衍射级次、增加光栅有效刻线总数，这也是郭守敬望远镜、高端光刻机光栅系统的核心设计逻辑。

3 实验与仿真设计

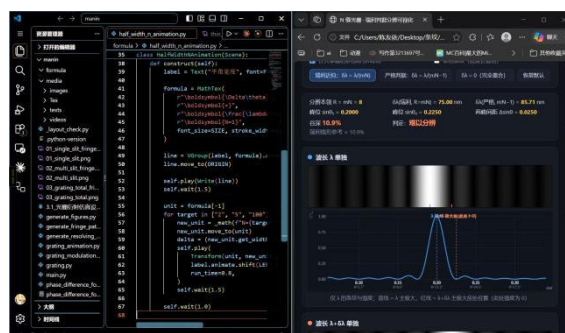
3.1 光栅衍射仿真设计

3.1.1 仿真目的

本项目依托数值仿真技术，对单缝衍射、多缝干涉、光栅缺级、亮纹半角宽度及色分辨本领等一系列核心光学现象进行精准复现。通过系统性的参数调控，直观验证理论公式的内在规律，为科普视频动画的制作提供精准、可量化且来源清晰的物理数据，从源头上规避纯手绘动画可能造成的物理失真，确保科普内容的科学性与严谨性。

3.1.2 仿真工具与环境

仿真以 Python 科学计算生态为核心，并辅以专业光学计算软件进行数据交叉校验。具体工具链配置如下：采用 NumPy 库执行数值计算，Matplotlib 库完成可视化绘图；引入 Mathematica 进行公式推导验证及高精度曲线拟合。该仿真环境运行稳定，计算精度满足光学科普与教学应用需求，所输出的图像、曲线及帧序列均可直接用于视频渲染合成。



仿真代码界面

3.1.3 仿真参数设置

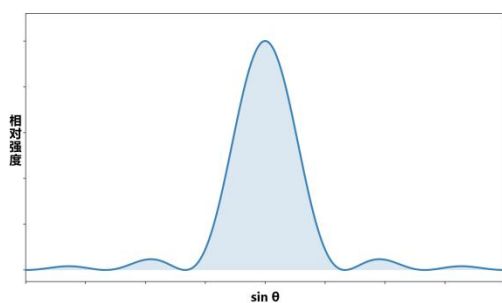
为简化，并保证不同工况间的数值可比性，仿真统一采用归一化波长 $\lambda = 1.0$ ，以 $\sin\theta$ 作为衍射角采样变量并覆盖 $[-1, 1]$ 全域，确保正负各级衍射图样完整呈现。可变核心参数包括单缝缝宽 a 、光栅常数 d 与光栅总缝数 N 。基准工况取 $a = 3.0$ 、 $d = 12.0$ 、 $N = 6$ ，对应 $d/a = 4$ ，此时每第 4 级干涉主极大恰与单缝衍射包络零点重合而发生缺级，调制结构清晰可辨。在此基础上另设多组对照工况，如 $d/a = 3, 4, 6, 8, 12$ 与衍射级次 $k = 1, 2, 3, 5, 8$ ，分别考察是缺级规律、亮纹半角宽度及色分辨本领随参数的变化。

3.1.4 仿真流程与方案

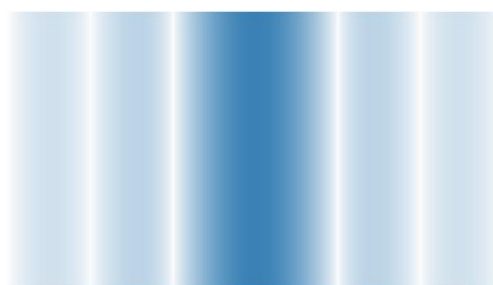
- 第一步，参数初始化与离散采样，对衍射角全域均匀取点，构建计算矩阵；
- 第二步，分别代入单缝衍射因子、多缝干涉因子公式，逐点计算光强数值；
- 第三步，叠加得到光栅衍射总光强，归一化处理消除亮度极值偏差；
- 第四步，固定部分参数、单一变量扫描，生成动态帧序列；
- 第五步，导出光强曲线、二维衍射图样、动态变化动画，用于视频素材制作。

3.1.5 仿真输出成果

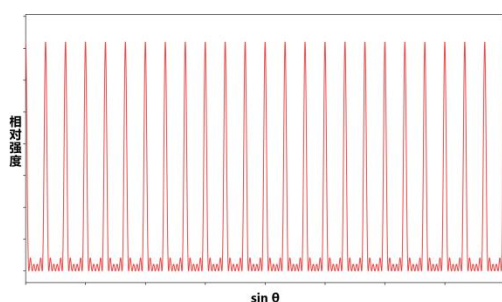
完整输出五类仿真成果：单缝衍射光强分布与条纹图样、多缝干涉精细条纹、光栅衍射缺级对比图样、不同缝数下半角宽度变化动画、多组参数下色分辨本领对比曲线。所有成果严格贴合物理公式，保证科普视频的科学性与严谨性。



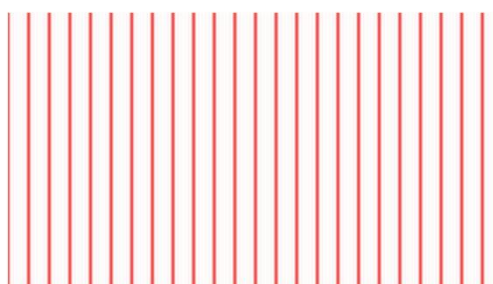
单缝衍射光强分布



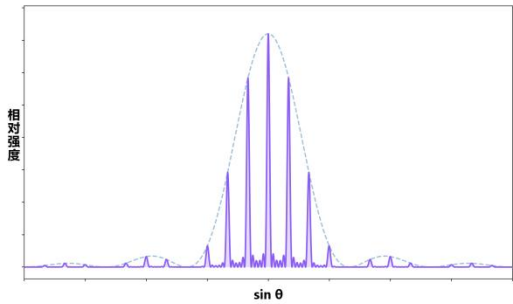
单缝衍射图样



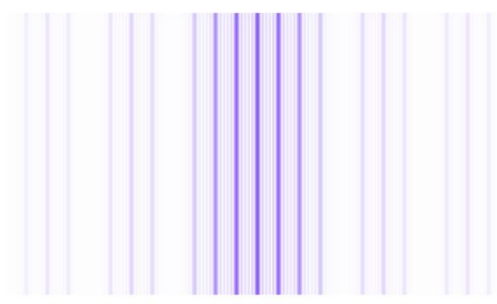
多缝干涉因子光强分布



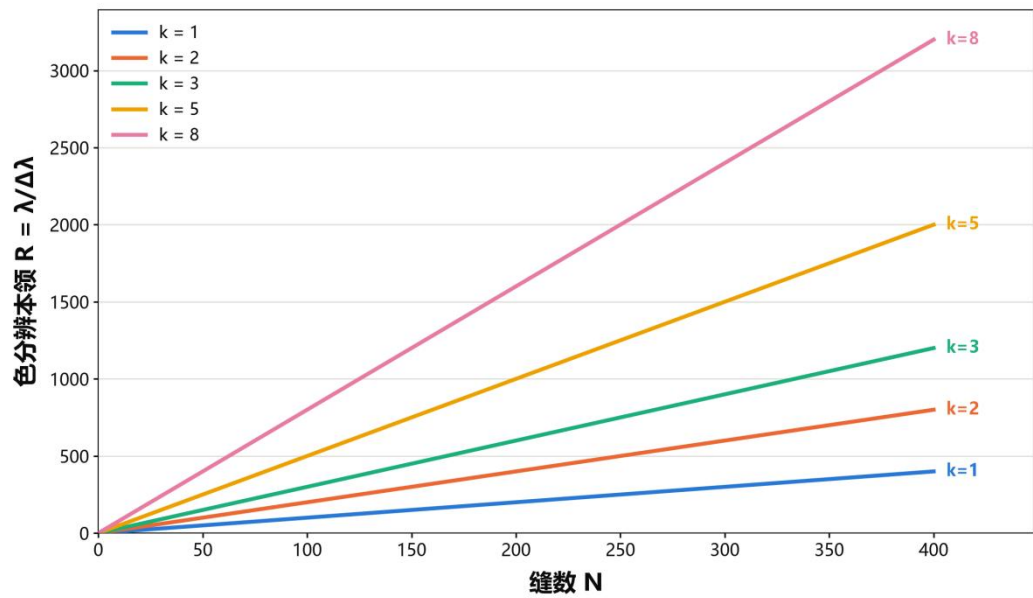
多缝干涉因子图样



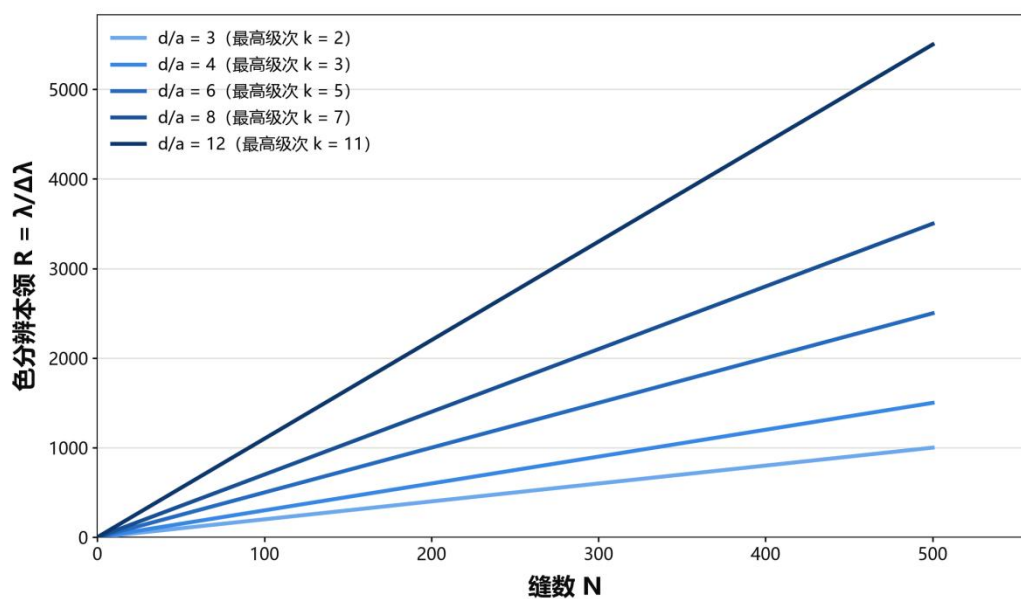
光栅总光强分布



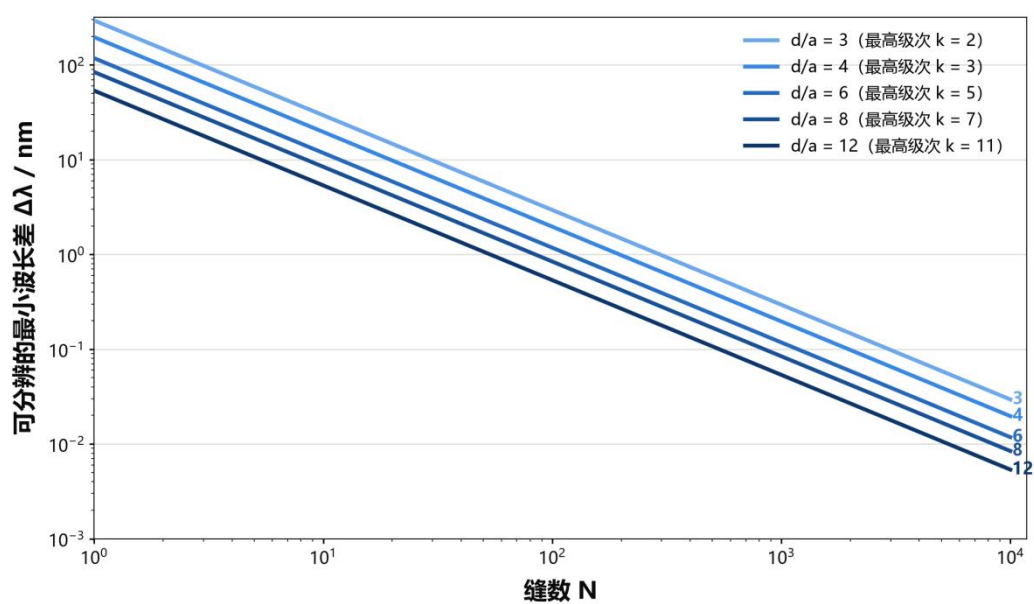
光栅总图样



光栅色分辨本领随缝数的变化



考虑缺级：最高可用级次下的色分辨本领



可分辨的最小波长差随缝数的变化

3.2 衍射现象验证实验

3.2.1 实验目的

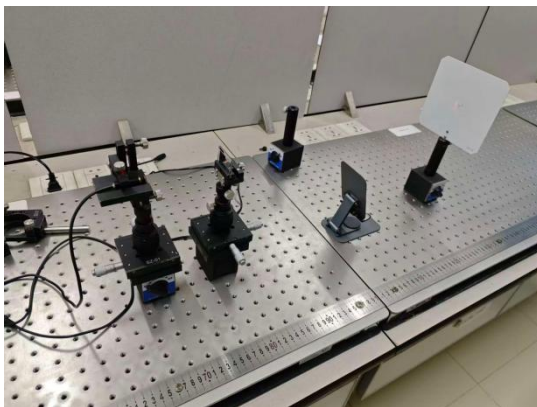
通过实物光学实验，直观验证仿真与理论推导结果，观测单缝衍射、光栅衍射、缺级现象的真实形貌，对比不同光栅参数下条纹粗细、亮度、分辨能力的差异，为视频实拍素材提供真实画面支撑，实现理论、仿真、实验三位一体的科普闭环。

3.2.2 实验器材

氦氖激光器（632.8nm 单色光源）、精密可调单狭缝、不同刻线密度标准光栅、光学光具座、白色观测光屏、高精度测距尺、游标卡尺。器材精度满足大学物理光学实验标准，可清晰观测各级衍射条纹。

3.2.3 实验内容与步骤

首先搭建同轴光学光路，校准激光器、衍射元件、光屏中心对齐，保证入射光垂直入射。单缝衍射实验中，逐步调节狭缝宽度，观测条纹宽窄、亮度变化规律并记录现象。光栅衍射实验中，更换不同光栅常数的光栅，观测规则明亮条纹，验证光栅分光特性。选用满足整数比值关系的光栅，清晰观测缺级现象，记录缺级级次。最后对比不同总缝数光栅的谱线精细度，定性验证半角宽度与分辨本领规律。



实验装置图

3.2.4 实验数据与误差分析

实验记录各组条纹间距、条纹粗细、缺级级次等观测数据，结合理论公式计算理论

值，对比实验误差。主要误差来源包括光路同轴度偏差、狭缝边缘衍射干扰、环境杂散光干扰、人眼读数偏差等。通过多次测量取平均值、遮光降噪、精密光路校准等方式降低误差，保证实验结果可靠。



$N=1$ 衍射光强分布



$N=2$ 衍射光强分布



$N=5$ 衍射光强分布



$N=100$ 衍射光强分布

4 视频设计与呈现

4.1 视频结构设计

本科普视频总时长约 173 秒，整体采用「大国重器悬念引入—科学历史溯源—分层原理拆解—核心特性解析—前沿应用升华—互动体验引流」的递进式叙事结构，节奏张弛有度，兼顾大众观赏性与学术严谨性，适配中小学科普、大学物理辅助教学双重场景。

4.1.1 开篇引入：大国重器与光栅价值（15s）

本段定位为悬念引流、价值铺垫。画面选取 ASML 光刻机精密光学系统特写、3nm 芯片蚀刻微观画面，聚焦大众熟知的高端制造场景，快速建立认知共鸣。通过文案点出光栅“有器无心”的行业痛点，点明光栅是高端光刻机的核心密码，抛出核心问题，引发观众探索兴趣，快速拔高项目立意，将基础光学原理与国家高端制造绑定。整体节奏紧凑、科技感极强，快速抓住观众注意力。

4.1.2 历史溯源：夫琅禾费与光栅发展（13s）

本段完成科学溯源，补齐理论发展脉络。画面采用复古画风，展示夫琅禾费人物肖像、1821 年光栅实验线稿，还原光栅理论的诞生背景。通过画面蒙太奇转场，从 19 世纪基础实验画面平滑过渡到现代光谱仪、光刻机、人造太阳等大国重器画面，串联起“百年科学理论支撑现代顶尖科技”的逻辑，让抽象原理具备历史厚重感与现实价值。

4.1.3 原理拆解：从单缝衍射到光栅衍射（62s）

本段为视频核心教学主体，分为单缝衍射、多缝干涉两个层级递进讲解。以生活中羽毛衍射实拍现象切入，降低入门门槛，再过渡到标准化物理模型。通过二维光路动画、三维光场演化、矢量叠加动态演示，逐帧拆解子波发射、光程差形成、振幅叠加的完整物理过程。同步配套公式分步推导动画，实现画面、配音、公式、物理意义四维同步，让零基础观众能看懂、有基础观众能学透彻，完美解决光学原理抽象难懂的痛点。

4.1.4 特性讲解：缺级、半角宽度与分辨本领（58s）

本段聚焦光栅三大核心特性，层层递进解析光栅高精度分光的本质。通过光强曲线动态叠加动画，直观演示单缝衍射调制、多缝干涉叠加形成缺级的全过程；通过参数动态调节动画，展示总缝数 N 变化对亮纹半角宽度的影响；依托瑞利判据可视化动画，通俗讲解光谱分辨本领的物理逻辑。所有抽象指标全部转化为肉眼可见的动态对比，知识点逻辑闭环、无断层。

4.1.5 应用升华：大国重器中的光栅技术（25s）

本段实现从理论回归工程、从知识升华价值。画面展示我国郭守敬望远镜 LAMOST、羲和号卫星、国产光刻机、人造太阳等自主科研重器，详细对应光栅技术的实际应用场景。文案升华基础科学的重要性，传递“小小光栅，撑起大国精密制造上限”的核心主旨，强化科技自信与科学探索精神，提升视频立意与传播价值。

4.1.6 结尾互动：沉浸式体验引导（收尾）

打破传统科普视频单向输出模式，增加交互式体验闭环。视频结尾展示自主开发的光栅衍射仿真网页，演示实时调节缝宽、缝数、波长参数，动态改变衍射图样的交互效果。屏幕展示专属二维码，引导观众自主上手实验，将被动观看转化为主动探索，延伸科普效果，提升作品创新性与实用性。

4.2 制作工具

本项目全流程采用专业影视、动画、仿真、剪辑、设计工具制作，分工明确、各司其职，保障视频画质精良、原理严谨、观感流畅，全套工具均为行业主流专业软件

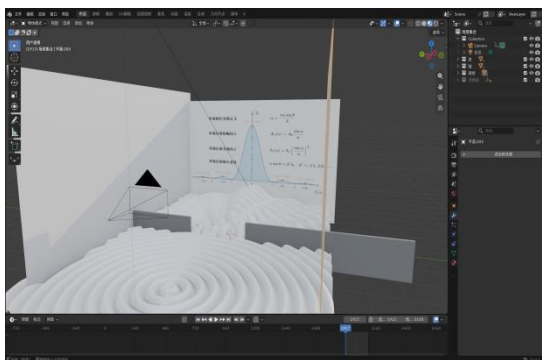
软件名称	具体工作内容
剪映专业版	全片剪辑、节奏把控、音画同步、画面合成
Blender 5.2.1	制作三维光场传播、立体衍射光路、光学设备场景建模
manim	二维动画、公式动效
Mathematica	数值仿真与科学素材生成
html	交互式仿真网页开发

4.3 动画实现

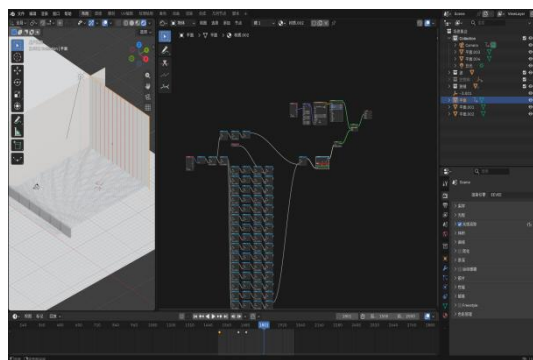
4.3.1 三维光场演化动画

本次实验基于 Blender 平台，以几何节点程序化计算 + 波纹修改器为核心技术方案，实现光栅衍射的三维光波场动态模拟。将二维接收平面离散为高精度网格，把电场振幅的数值映射为网格顶点的 Z 向位移，以曲面的起伏直观表征光场振幅的空间分布；同时配套理论公式对照与视角动画，分别完成单缝衍射、多缝光栅衍射的三维可视化

视频制作。衍射光场程序化生成 基于惠更斯 - 菲涅耳原理，通过 Blender 几何节点系统实现单缝衍射光场的数值计算：将接收平面离散为高密度网格，逐点计算电场复振幅，将振幅值线性映射为网格顶点的 Z 向位移，生成起伏的三维波场曲面。场景背景同步配套单缝衍射的理论公式与光强分布曲线。多缝光栅衍射三维模拟通过几何节点实现多缝衍射光场的相干叠加计算：对每条狭缝分别计算单缝衍射场，再引入相邻狭缝间的相位差，将所有狭缝的衍射场相干叠加得到总光场；接收面网格的顶点位移同步反映总电场的振幅分布，曲面上自然呈现出等间距的干涉主极大结构，主极大之间分布着次级极小与次极大，与光栅衍射的理论特征完全吻合。从三维视角可直观观察到：狭缝数量越多，干涉主极大的形态越锐利，主极大之间的暗区范围越宽，准确复现了多缝衍射的条纹的特性。



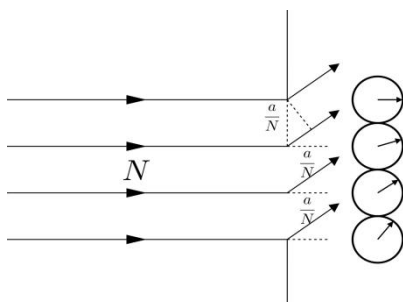
Blender 建模界面



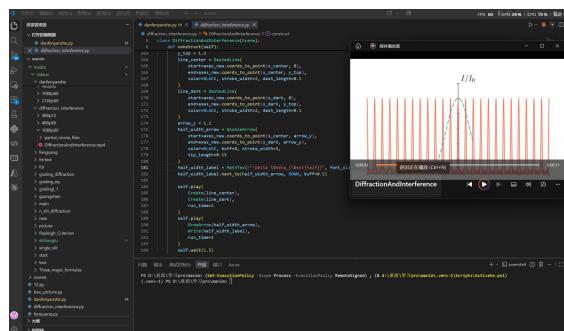
Blender 工作流

4.3.2 二维衍射光路动画

二维衍射光路动画依托 Manim 动画引擎进行绘制，严格参照大学物理教材标准建立标准化光学光路模型，图形线条规范统一，物理标注严谨准确。动画采用路径描边的逐帧生成方式，动态还原平行光入射狭缝、子波发射、衍射光传播的完整光路过程，动态标注缝宽 a 、光栅常数 d 、衍射角 θ 、光程差、相位差等关键物理量。动画使用色彩区分入射光、衍射光以及相干叠加区域，配合矢量箭头直观表现各子波的振动相位变化，将单缝内部子波分解、多缝之间光程差形成的抽象过程可视化。动画节奏与配音讲解完全同步，讲解到对应知识点时同步展示光路变化，规避静态示意图难以表现动态传播过程的短板，适配课堂教学的演示需求。



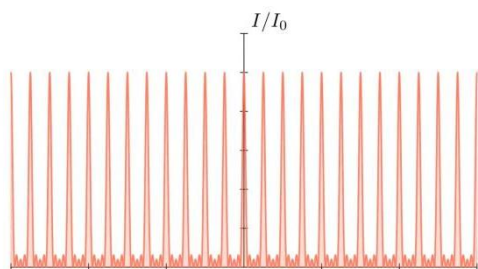
单缝衍射光路动画效果图



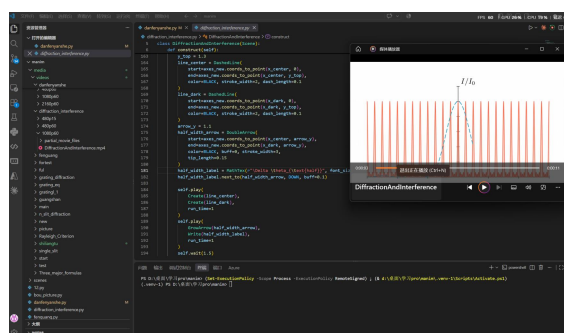
单缝衍射光路动画 manim 代码界面

4.3.3 光强分布动画

光强分布动画全部采用 Python 数值仿真输出的真实物理数据作为绘制依据，杜绝手绘动画带来的物理失真问题。借助 Manim 实现光强曲线逐点描边的生成动画，依次演示单缝衍射包络、多缝干涉光强曲线，再通过图像变换展示二者相乘得到光栅衍射总光强的完整过程。针对缺级、半角宽度、色分辨本领等核心特性，设置分屏对照动画，动态修改缝数、光栅常数、缝宽等参数，直观表现参数改变对光强曲线、谱线锐度带来的变化。公式采用分步弹出解析的动画形式，每一项公式出现均对应光强图像的物理变化，例如展示半角宽度时，同步在光强曲线上绘制主极大中心与相邻暗纹的标记虚线、双向箭头与符号标注，将抽象数学符号和光强分布图像一一对应，帮助受众理解公式背后的物理内涵，而不是单纯记忆数学表达式。



多缝干涉光强分布效果图



多缝干涉光强分布 manim 代码界面

4.3.4 公式推导动画

依托仿真真实数据生成光强曲线，采用逐点描边动画动态绘制曲线成型过程，杜绝手

绘失真问题。公式采用分步出现、逐项解析的动画逻辑，每一步推导对应物理图像。缺级叠加、半角宽度对比、分辨本领变化均采用分屏对照动画，参数变化直观可见，让观众清晰理解公式背后的物理意义，而非死记公式。

$$\beta = \frac{\pi}{\lambda} d \sin \theta$$

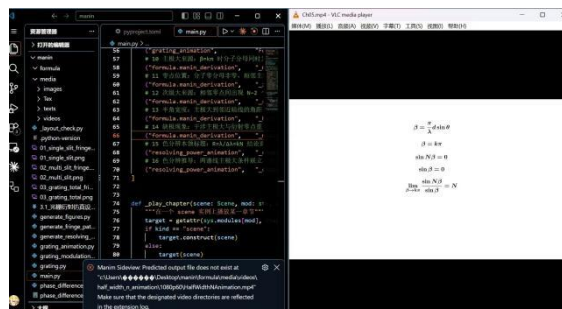
$$\beta = k\pi$$

$$\sin N\beta = 0$$

$$\sin \beta = 0$$

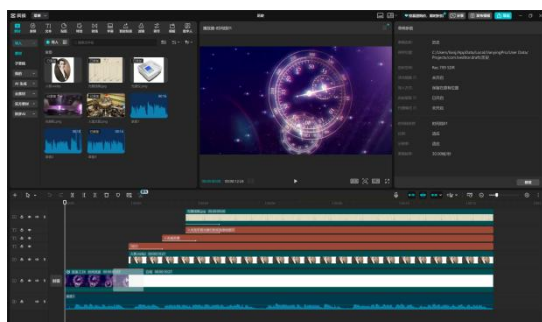
$$\lim_{\beta \rightarrow k\pi} \frac{\sin N\beta}{\sin \beta} = N$$

多缝干涉公式效果图

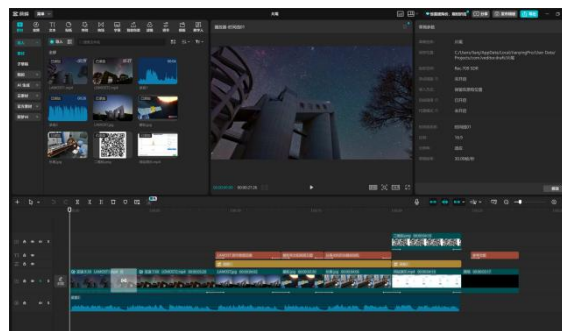


多缝干涉公式制作代码界面

4.3.5 剪辑动画



光栅衍射历史部分剪辑界面



光栅衍射片尾部分剪辑界面

4.4 多媒体素材整合

4.4.1 素材分类与规整

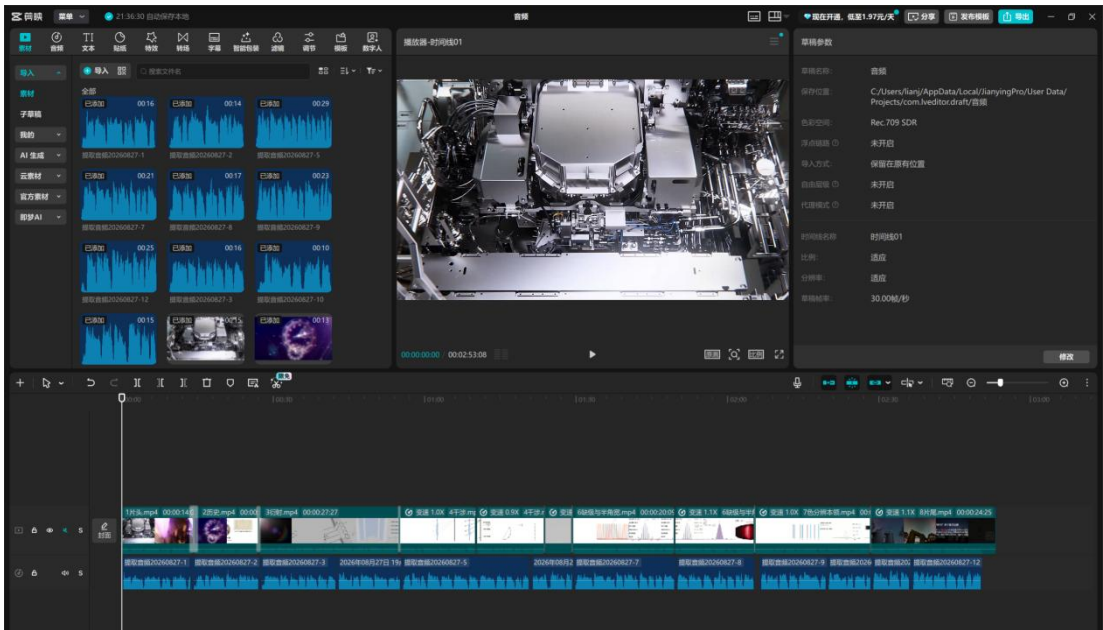
项目海量素材进行标准化分类管理，分为实拍素材、三维动画素材、二维动效素材、图文字幕素材、音频素材五大类。实拍素材包含光刻机官方宣传片、LAMOST 望远镜延时摄影、羽毛衍射实拍、光学实验实拍；动画素材包含全套光路、矢量叠加、光强曲线、参数变化动画；图文素材包含科学人物画像、设备介绍卡片、公式模板、标题字幕、互动二维码；音频素材包含专业配音、科技转场音效、沉浸式背景配乐。所有素材统一分辨率、统一色彩规范，便于剪辑整合。



剪辑草稿界面

4.4.2 剪辑节奏把控

差异化分段把控视频节奏：开篇与结尾升华段落节奏明快、镜头切换利落，增强视觉冲击力；原理推导、公式讲解、特性分析段落节奏舒缓，延长关键画面停留时间，适配观众理解吸收速度；转场方式精细化区分，历史段落用淡入淡出、原理递进用元素承接转场、科技展示用快切闪转，整体节奏层次分明、观感舒适。

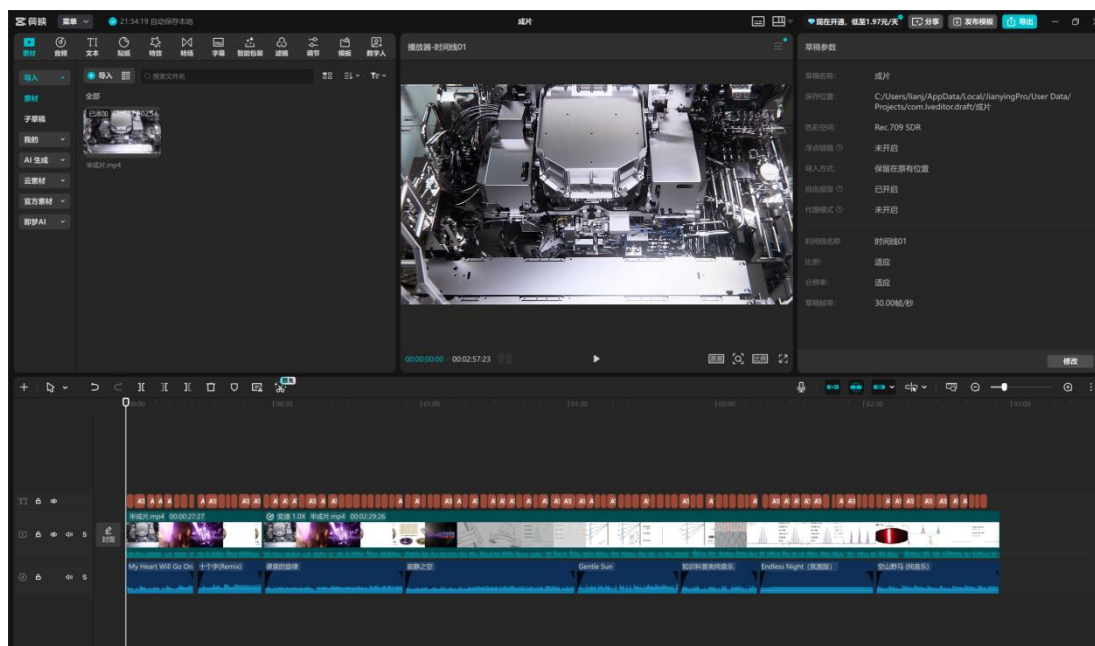


汇总各段加配音剪辑界面

4.4.3 音画同步与包装规范

实现配音、字幕、动画、音效四维精准同步，关键词、核心结论同步高亮提示。全片统一科技蓝主色调，历史段落复古暖黄、应用段落高亮通透，色彩分区明确。字幕严

格遵循学术规范，物理符号斜体、公式居中、重点结论高亮，画面干净高级、专业性强。全片 1080P 高清渲染、帧率稳定，画质达到竞赛级作品标准。



加字幕背景音乐剪辑界面

5 创新点与总结

5.1 创新点

5.1.1 教学形式创新：科普动画结合前沿重器应用

突破传统物理科普“重理论、轻应用、无场景”的弊端，将抽象波动光学原理与我国光刻机、巡天望远镜、人造太阳等大国重器深度绑定，让基础物理知识落地到高端科研与工业场景。采用三维立体动画+二维教学动效+真实实拍素材的多元呈现形式，兼顾趣味性、观赏性与学术严谨性，既适合大众科普传播，也可作为高校物理课堂、物理实验的标准化辅助教学资源。

5.1.2 内容设计创新：基础原理到工程应用的递进呈现

内容架构完全遵循人类认知规律，构建「生活现象—科学历史—基础原理—核心特性—工程应用」的完整知识闭环。从具象实拍现象切入，逐层拆解单缝衍射、多缝干涉、光栅衍射、缺级、分辨本领等难点知识点，层层递进、逻辑无断层，完整覆盖大学物理光栅衍射全部考点，同时拓展前沿工程应用，实现“课内知识+课外前沿”的双向拓展，内容深度与适配广度远超普通科普作品。

5.1.3 交互设计创新：参数可调的沉浸式衍射体验

打破传统科普视频单向传播的局限性，配套开发可自主操作的光栅衍射交互网页。观众可手动调节缝宽、光栅常数、总缝数、入射光波长等核心参数，实时观测衍射图样、光强曲线的动态变化，自主验证理论规律。将被动观看升级为主动探究式学习，弥补视频无法个性化演示参数变量的短板，极大提升科普学习效果，具备极强的教学实用价值与创新性。

5.2 总结

本项目完整完成了光栅衍射科普微视频的理论研究、仿真实验、动画制作、剪辑包装、交互开发全流程工作。作品系统梳理了惠更斯-菲涅耳原理、夫琅禾费衍射、单缝衍射、多缝干涉、光栅缺级、半角宽度、色分辨本领的全套理论体系，通过高精度仿真与可视化动画，彻底解决光栅衍射原理抽象、难点难懂的教学痛点。

项目将基础物理理论与国家高端精密制造、前沿天文科研、可控核聚变等大国工程相结合，既普及了专业光学知识，又展现了基础科学对国家科技发展的支撑作用，具备良好的科普价值、教学价值与思政价值。成品视频画面精良、逻辑严谨、节奏舒适、立意深刻，配套交互工具实用性极强，可广泛应用于校园物理教学、线上科普传播、科创竞赛展示等场景。

本次项目实践也积累了光学可视化、科学动画制作、科普内容设计的完整经验，后续可进一步拓展衍射光学、光谱技术等相关科普内容，完善光学科普体系，持续优化交互体验，让基础科学知识以更通俗、更直观、更有趣的形式走进大众与课堂。

6 参考资料

- [1] 赵凯华, 钟锡华. 光学[M]. 北京大学出版社, 2004.
- [2] 梁铨廷. 物理光学[M]. 电子工业出版社, 2012.
- [3] 张三慧. 大学物理学：波动与光学分册[M]. 清华大学出版社, 2019.
- [4] 郁道银, 谈恒英. 工程光学[M]. 机械工业出版社, 2016.
- [5] 中国科学院国家天文台. 郭守敬望远镜（LAMOST）科学成果简介[EB/OL].
- [6] 周炳琨. 激光原理[M]. 国防工业出版社, 2014.
- [7] 约瑟夫·W·古德曼. 傅里叶光学导论[M]. 电子工业出版社, 2020.

附件 1：组员分工

成员序号	负责模块
成员 1	策划、文案、配音、剪辑、项目报告书
成员 2	二维图形动画制作
成员 3	三维动画制作、PPT
成员 4	公式推导动画制作
成员 5	初步文案思路

附件 2：全部代码

(1)视频动画部分

```
danfenyanshe.py
diffraction_interference.py
shiliangtu.py
guangshan.py
single_slit.py
```

#danfenyanshe.py

<pre>from manim import * import numpy as np def MidpointArrow(start: np.ndarray, end: np.ndarray, color: str = BLUE, stroke_width: float = 4, tip_length: float = 0.2, tip_width: float = 0.1, **kwargs) -> VGroup: # 1. 绘制主线段（不带箭头） line = Line(start, end, color=color, stroke_width=stroke_width, **kwargs) # 2. 计算中点 mid = (start + end) / 2 # 3. 计算方向（从起点指向终点） direction = end - start norm = np.linalg.norm(direction)</pre>	<pre> return VGroup(line) unit_dir = direction / norm # 4. 创建等腰三角形箭头（顶点朝 上，即 y 轴正方向） tip = Polygon(np.array([0, tip_length / 2, 0]), # 尖端顶点 np.array([-tip_width / 2, - tip_length / 2, 0]), # 左下 np.array([tip_width / 2, -tip_length / 2, 0]), # 右下 color=color, fill_color=color, fill_opacity=1, stroke_width=0, # 无描边，纯 填充) # 5. 旋转使尖端指向 unit_dir angle = np.arctan2(unit_dir[1], unit_dir[0]) - np.pi / 2 tip.rotate(angle,</pre>
---	--

```

# 6. 将箭头放置到中点
tip.move_to(mid)

# 7. 组合返回
return VGroup(line, tip)

class GratingDerive(Scene):
    def construct(self):
        self.camera.background_color =
"white"

        #入射
        d = 0.5

        inlight = VGroup()
        for i in range(-2, 2):
            light = MidpointArrow(
                start=LEFT * 4 + (i+0.65) *
d * DOWN,
                end=LEFT * 1 + (i+0.65) *
d * DOWN,
                color=BLACK,
stroke_width=4
            )
            inlight.add(light)

            N_label = Text("N 条光
",font="SimSun",
font_size=25,color=BLACK).next_to(inligh
t[0][1], DOWN, buff=0.5)
            inlight.add(N_label)

            cond1 = Text("平行光
",font="SimSun",
font_size=24,color=BLACK).next_to(inligh
about_point=ORIGIN)

            inlight.move_to(ORIGIN + LEFT *
5)

            self.play(FadeIn(inlight))
            self.play(Write(cond1))
            self.wait(0.5)
            self.play(FadeOut(cond1))

            #单缝绘制
            fen=VGroup()

            slit = Line(
                start= 6 * UP,
                end= 0.75 * UP,
                color=BLACK, stroke_width=8
            )
            fen.add(slit)

            slit= Line(
                start= -6 * UP,
                end= -0.75 * UP,
                color=BLACK, stroke_width=8
            )
            fen.add(slit)

            self.play(FadeIn(fen))

            dbarrow=DoubleArrow(
                start=0.75*UP,
                end=0.75*DOWN,
                color=BLACK,
stroke_width=4,buff=0
            )

```

<pre> t[0][0], UL, buff=0.5) a_lable=MathTex(r"a", font_size=30,color=BLACK).next_to(dbarrow, RIGHT, buff=0.1) self.play(FadeIn(dbarrow,a_lable)) self.wait(0.5) self.play(FadeOut(dbarrow,a_lable)) self.play(inlight.animate.move_to(ORIGIN+LEFT*1.5)) bacegroup=VGroup(inlight,fen) self.play(bacegroup.animate.shift(LEFT*3).scale(2)) helpline=VGroup() for i in range(4): slit= DashedLine(start=inlight[i][0].get_end(), end=inlight[i][0].get_end() +RIGHT*0.8, color=BLACK, stroke_width=4) helpline.add(slit) for i in range(3): lable= MathTex(r"\frac{a}{N}", font_size=30,color=BLACK).next_to(helpline[i], DOWN, buff=0.1) helpline.add(lable) self.play(FadeIn(helpline)) phase=VGroup() for i in range(4): </pre>	<pre> phase_circle=Circle(radius=0.5 , color=BLAC fill_color=W HITE, fill_opacity=0).move_to(helpline[i].g et_end()).shift(RIGHT*1.0) phase.add(phase_circle) for i in range(4): phase_arrow= Vector(direction=RIGHT* 0.5, color=BLACK, stroke_width=6).move_to(phase[i].get_center()+R IGHT*0.25) phase.add(phase_arrow) cod2=Text("相位", font_size=24,color=BLACK).next_to(phase [0], UP, buff=0.5) #出射 outlight=VGroup() for i in range(2): slit=Arrow(start=inlight[i][0].get_end(), end=inlight[i][0].get_end()+RIGHT*1. 3, color=BLACK, stroke_width=4,buff=0,tip_length=0.2) </pre>
--	--

```

        self.play(FadeIn(phase,outlight))
#光的角度变换
        helpline_right_line=DashedLine(
            start=inlight[0][0].get_end(),
            end=inlight[1][0].get_end(),
            color=BLACK, stroke_width=4
        )
        helpline_aside_line=helpline[0]
        right_angle =
RightAngle(outlight[1],
helpline_aside_line, length=0.2,
color=BLACK,quadrant=(-1,-1))

        theta_label= MathTex(r"\theta",
font_size=25,color=BLACK).next_to(helpline[1], UP, buff=0.05)

        self.play(FadeIn(helpline_aside_line))
self.play(FadeIn(helpline_right_line,theta_label),helpline_aside_line.animate.rotate(0.9273,IN,about_point=helpline_aside_line.get_start()),helpline[4].animate.shift(LEFT*0.7),outlight[0].animate.rotate(0.6435,OUT,about_point=outlight[0].get_start()),
        outlight[1].animate.rotate(0.6435,OUT,about_point=outlight[1].get_start()),

        #出射角度
phase[4].animate.rotate(0.0,OUT,about_point=phase[4].get_start()),

phase[5].animate.rotate(0.3,OUT,about_point=phase[5].get_start()),

phase[6].animate.rotate(0.6,OUT,about_p

```

```

        outlight.add(slit)

phase[7].animate.rotate(0.9,OUT,about_point=phase[7].get_start()),

        )
        self.play(FadeIn(right_angle))
        self.wait(1)
        self.play(FadeOut(phase))
        self.wait(1)

        # 光强曲线可视化
        a = 1.0
        d = 3.0
        wavelength = 1.0
        I0 = 1.0

        def intensity_func(sintheta, N):

            alpha = np.pi * a * sintheta / wavelength

            beta = np.pi * d * sintheta / wavelength

            with
np.errstate(divide='ignore',
invalid='ignore'):

                diff =
np.square(np.sin(alpha) / alpha)

                diff[alpha == 0] = 1.0

                inter = np.square(np.sin(N * beta) / np.sin(beta))

                # 主极大处
                inter[np.abs(beta % np.pi) < 1e-8] = N**2

                inter[np.isnan(inter)] = 0

```

<pre> oint=phase[6].get_start()), axes = Axes(x_range=[-2, 2, 0.2], y_range=[0, 120, 20], axis_config={"include_numbers": False}, x_length=10, y_length=5, color=BLACK,) x_label = axes.get_x_axis_label(r"\sin\theta") y_label = axes.get_y_axis_label(r"I/I_0", edge=UP, direction=UP) N_values = [1] colors = [BLACK] curves = VGroup() labels = VGroup() for i, N in enumerate(N_values): x_vals = np.linspace(-2, 2, 200) y_vals = intensity_func(x_vals, N) </pre>	<pre> return I0 * diff * inter curve = axes.plot_line_graph(x_vals, y_vals, line_color=colors[i], add_vertex_dots=False, stroke_width=2,) label = Text(f"N = {N}", color=colors[i], font_size=30).to_corner(UP + RIGHT) if i > 0: label.next_to(labels[-1], DOWN, buff=0.2) curves.add(curve) labels.add(label) self.play(Create(axes), Write(x_label), Write(y_label)) self.wait(1) for i, N in enumerate(N_values): self.play(Create(curves[i]), Write(labels[i]), run_time=2.0) self.wait(1)""" </pre>
--	---

#diffraction_interference.py

<pre> from manim import * import numpy as np class DiffractionAndInterference(Scene): </pre>	<pre> # 坐标轴（黑色） title_text = "光栅衍射：单缝衍射 与多缝干涉" title = Text(title_text, </pre>
--	--

```

def construct(self):
    self.camera.background_color = WHITE
    # 物理参数
    d_over_a = 4.0    # 光栅常数 / 缝宽
    N = 5             # 缝数
    # 强度函数
    def single(x):
        # 单缝衍射因子:  $I_{\text{single}} = (\sin(\pi x)/(\pi x))^2 = \text{sinc}(x)^2$ 
        return np.sinc(x) ** 2
    def inter(x):
        # 多缝干涉因子:  $I_{\text{inter}} = (\sin(N\gamma)/(N \sin\gamma))^2$ ,  $\gamma = \pi(d/a)x$ 
        gamma = np.pi * d_over_a * x
        with np.errstate(divide='ignore', invalid='ignore'):
            result = np.where(
                np.abs(np.sin(gamma)) < 1e-12,
                1.0, # 主极大位置
                (np.sin(N * gamma) / (N * np.sin(gamma))) ** 2
            )
        return result
    def total(x):
        return single(x) * inter(x)
    # 单缝衍射包络线 (虚线)
    single_solid = axes.plot(single,
font="SimHei", color=WHITE,
font_size=32)
    title.to_edge(UP, buff=0.4)
    axes = Axes(
        x_range=[-3, 3, 1],
        y_range=[0, 1.2, 0.2],
        axis_config={"color": BLACK, "stroke_width": 2},
        tips=False,
    )
    axes.next_to(title, DOWN, buff=0.8)
    axes_labels = axes.get_axis_labels(
        y_label=MathTex(r"I/I_0", color=BLACK),
    )
    title = Text("光栅衍射：单缝衍射与多缝干涉", font_size=36, color=BLACK).to_edge(UP)
    self.play(Write(axes), Write(axes_labels))
    #self.wait(0.5)
    # 曲线
    x_range = [-3, 3, 0.001]
    # 1. 单缝衍射包络线 (虚线)
    self.play(Create(single_curve), run_time=2)
    self.wait(1)

```

<pre> x_range=x_range, color=GRAY, stroke_width=3) single_curve = DashedVMObject(single_solid, num_dashes=50,dashed_ratio=0.7) single_curve.set_color(BLUE_D).set _stroke(width=4) # 多缝干涉因子 inter_curve = axes.plot(inter, x_range=x_range, color=RED, stroke_width=2) # 总强度曲线 total_curve = axes.plot(total, x_range=x_range, color=PURPLE, stroke_width=4) # 动画流程 # 2. 多缝干涉因子 #self.play(FadeIn(inter_curve), run_time=3) #area = axes.get_area(inter_curve, x_range=[-3, 3], color=RED, opacity=0.3) #self.play(FadeIn(area)) #self.wait(1) # 之后如果曲线变化 (例如改变参数), 填充会自动重绘 self.wait(0.3) # 2. 计算水平拉伸比例 # x 范围 [-0.2,0.2] 长度 0.4 </pre>	<pre> # 3. 展示乘法关系 (文字说明) multiply_note = Text("单缝衍射因子 × 多缝干涉因 子 → 光栅总强", fot_size=24, color=BLACK).to_edge(DOWN) #self.play(Write(multiply_note)) #self.wait(1) # 4. 平滑变形: 干涉曲线复制变 形为总强度曲线, # 同时单缝包络线和干涉曲线 变淡, 突出总强度 self.play(ReplacementTransform(graph_area_grou p, total_curve),) self.play(total_curve.animate.set_opacity(0.7)) # 聚焦半角宽度: 横向拉伸动画 # 1. 淡出其他曲线和标签 fade_out_others = VGroup(single_curve, axes_labels) self.play(FadeOut(fade_out_others), run_time=1) total_curve_correct = axes_new.plot(total, </pre>
--	--

<pre> # 希望中心区域放大, 拉伸倍数 k = 6 / 0.4 = 15 k = 15 # 4. 第二步: 创建新坐标轴并变换 axes_new = Axes(x_range=[-0.2, 0.2, 0.05], y_range=[0, 1.2, 0.2], axis_config={"color": BLACK, "stroke_width": 2}, tips=False,) axes_new.next_to(title, DOWN, buff=0.8) # 3. 第一步: 仅水平拉伸曲线 self.play(total_curve.animate.apply_function(lambda p: np.array([p[0] * k, p[1], p[2]]),), Transform(axes, axes_new), run_time=2, rate_func=smooth) self.wait(0.3) arrow_y = 1.1 half_width_arrow = DoubleArrow(start=axes_new.coords_to_point(x_center, arrow_y), </pre>	<pre> x_range=[-0.2, 0.2, 0.0001], color=PURPLE, stroke_width=4,) # 5. 标记半角宽度 x_center = 0 x_dark = 0.05 y_top = 1.3 line_center = DashedLine(start=axes_new.coords_to_point(x_center, 0), end=axes_new.coords_to_point(x_center, y_top), color=BLACK, stroke_width=2, dash_length=0.1) line_dark = DashedLine(start=axes_new.coords_to_point(x_dark, 0), end=axes_new.coords_to_point(x_dark, y_top), color=BLACK, stroke_width=2, dash_length=0.1) </pre>
--	--

```

        end=axes_new.coords_to_point(
            x_dark, arrow_y),
        color=BLACK, buff=0,
        stroke_width=3,
        tip_length=0.15
    )
    half_width_label =
    MathTex(r"\Delta \theta_{\text{half}}",
    font_size=30, color=BLACK)
    half_width_label.next_to(half_width_arrow, DOWN, buff=0.1)

    self.play(
        Create(line_center),
        Create(line_dark),
        run_time=1
    )
    self.play(
        GrowArrow(half_width_arrow),
        Write(half_width_label),
        run_time=1
    )
    self.wait(1.5)

```

#shiliangtu.py

```

from manim import *
import numpy as np

def move_to_arc_tangent(
    arc: Arc,

```

```

        current_dir = mobject.get_right() -
        mobject.get_center()

        elif np.array_equal(aligned_axis, UP):
            current_dir = mobject.get_top() -
            mobject.get_center()

        else:

```

<pre> mobject: Mobject, buff: float = 0.5, t: float = 0.5, align_axis: np.ndarray = RIGHT) -> Mobject: center = arc.arc_center radius = arc.radius start_angle = arc.start_angle total_angle = arc.angle theta = start_angle + t * total_angle point_on_arc = center + radius * np.array([np.cos(theta), np.sin(theta), 0]) radial_dir = (point_on_arc - center) / radius tangent_dir = np.array([-np.sin(theta), np.cos(theta), 0]) tangent_dir = tangent_dir / np.linalg.norm(tangent_dir) if total_angle < 0: tangent_dir = -tangent_dir new_point = point_on_arc + buff * radial_dir mobject.move_to(new_point) if np.array_equal(align_axis, RIGHT): </pre>	<pre> raise ValueError("align_axis 必须 是 RIGHT 或 UP") if np.linalg.norm(current_dir) < 1e-9: current_dir = np.array([1.0, 0.0, 0.0]) else: current_dir = current_dir / np.linalg.norm(current_dir) cross = np.cross(current_dir, tangent_dir) dot = np.dot(current_dir, tangent_dir) angle_to_rotate = np.arctan2(cross, dot) mobject.rotate(angle_to_rotate, about_point=mobject.get_center()) return mobject class GratingDerive(Scene): def construct(self): self.camera.background_color = "white" x_range = [-5, 5, 1] y_range = [-5, 5, 1] axis_length = 8 </pre>
<pre> axis = Axes(x_range=x_range, y_range=y_range, x_length=axis_length, y_length=axis_length, axis_config={"color": WHITE, </pre>	<pre> arrow.add(arr_each) self.play(Write(arrow[0]), Write(arrow[1]), run_time=2/n) help_line=DashedLine(start=arrow[0].get_start(), end=arrow[0].get_end(), </pre>

<pre> "stroke_width": 0},) self.play(Write(axis)) arrow = VGroup() angle = -PI * 0.7 n = 5 ave_angle = angle / n r = 3 for i in range(n): x1 = r * np.sin(ave_angle * i + PI) y1 = r * np.cos(ave_angle * i + PI) x2 = r * np.sin(ave_angle * (i + 1) + PI) y2 = r * np.cos(ave_angle * (i + 1) + PI) arr_each = Arrow(start=axis.coords_to_point(x1, y1), end=axis.coords_to_point(x2, y2), color=BLACK, stroke_width=4, buff=0) </pre>	<pre> color=BLACK, stroke_width=2, buff=0) help_line.scale(2,about_point=arro w[0].get_start()) self.play(Write(help_line), run_time=0.5) delta_label = MathTex(r"\mathbf{\delta}=2\mathbf{\be ta}", font_size=30, color=BLACK) delta_label.next_to(help_line, UP,buff=0.1).shift(RIGHT*1.3+DOWN*0.0 5) angle=Angle(arrow[0], arrow[1], radius=0.5, color=BLACK, stroke_width=2) self.play(Write(delta_label), Write(angle), run_time=1,lag_ratio=0.0) for i in range(n-2): self.play(Write(arrow[i+2]), run_time=1/n) R = VGroup() for i in range(n): </pre>
<pre> slit = Line(start=ORIGIN, end=arrow[i].get_start(), color=BLACK, </pre>	<pre> # 获取两条线的方向向量 dir1 = R[i].get_end() - R[i].get_start() dir2 = R[i+1].get_end() - R[i+1].get_start() angle1 = np.arctan2(dir1[1], </pre>

<pre> stroke_width=2, buff=0) R.add(slit) slit = Line(start=ORIGIN, end=arrow[n-1].get_end(), color=BLACK, stroke_width=2, buff=0) R.add(slit) self.play(Write(R), run_time=1, lag_ratio=0.0) # 显示角度标记 (使用 Angle 对 象) angle_group = VGroup() for i in range(n): ang = Angle(R[i], R[i+1], radius=0.5, color=BLACK, stroke_width=2) angle_group.add(ang) self.play(Write(angle_group), run_time=1, lag_ratio=0.0) # 重点修改: 利用 R 直接计算 弧线, 放置 2β 标签 beta_group = VGroup() for i in range(n): </pre>	<pre> dir1[0]) angle2 = np.arctan2(dir2[1], dir2[0]) delta = angle2 - angle1 # 调整为较小夹角, 保证弧线 覆盖两条线之间的区域 if delta < 0: start_angle = angle2 span = -delta else: start_angle = angle1 span = delta # 创建与 Angle 半径一致的 Arc 对象 (只用作位置参考, 不显示) arc = Arc(radius=0.5, start_angle=start_angle, angle=span) beta = MathTex(r"2\mathbf{\beta}", font_size=25, color=BLACK) move_to_arc_tangent(arc, beta, buff=0.3, t=0.5, align_axis=UP) beta_group.add(beta) self.play(Write(beta_group), run_time=1, lag_ratio=0.0) </pre>
<pre> # 总箭头 last_end = arrow[-1].get_end() tol_arrow = Arrow(start=axis.coords_to_point(0, - </pre>	<pre> # 将剩余箭头变换为直线首尾相 接 (竖直方向) # 计算每个箭头的长度 lengths = [arrow[i].get_length() for i in range(n)] </pre>

<pre> r), end=last_end, color=RED, stroke_width=4, buff=0, tip_length=0.2) self.play(Write(tol_arrow), run_time=0.75) self.wait(1) # 修改：移除所有辅助元素，但 保留 tol_arrow to_fade = VGroup(R, # 从原点发出的半径线 angle_group, # 相邻半径 线间的角度弧线 beta_group, # 2β 标签 angle, # 最初的小角 度标记 help_line, # 虚线辅助线 delta_label # δ=2β 标签 # 注意：移除了 tol_arrow, 因为它需要保留并变形) self.play(FadeOut(to_fade), run_time=1) self.wait(0.5) </pre>	<pre> total_len = sum(lengths) # 竖直排列的起点：使直线居中 于原点 start_y = -total_len / 2 # 为每个箭头生成目标位置（竖 直向上，首尾相接） for i in range(n): start = np.array([0, start_y, 0]) end = start + np.array([0, lengths[i], 0]) # 沿 y 轴正方向 arrow[i].generate_target() arrow[i].target.put_start_and_e nd_on(start, end) start_y += lengths[i] # 更新 下一个箭头的起点 # 为 tol_arrow 设置目标状态 (起点 = 第一个箭头目标起点，终点 = 最后一个箭头目标终点) tol_arrow.generate_target() tol_arrow.target.put_start_and_end_on(arrow[0].target.get_start(), arrow[-1].target.get_end()) </pre>
<pre> # 播放动画：所有箭头和 tol_arrow 同时移动到各自目标 self.play(*[MoveToTarget(arrow[i]) for i in range(n)], </pre>	

```

        MoveToTarget(tol_arrow),
        run_time=2,
        rate_func=smooth
    )
    self.wait(1)

```

#guangshan.py

```

from manim import *
import numpy as np

def MidpointArrow(
    start: np.ndarray,
    end: np.ndarray,
    color: str = BLUE,
    stroke_width: float = 4,
    tip_length: float = 0.2,
    tip_width: float = 0.1,
    **kwargs
) -> VGroup:
    # 1. 绘制主线段（不带箭头）
    line = Line(start, end, color=color,
                stroke_width=stroke_width, **kwargs)

    # 2. 计算中点
    mid = (start + end) / 2

    # 3. 计算方向（从起点指向终点）
    direction = end - start
    norm = np.linalg.norm(direction)
    if norm == 0: # 如果起点终点重

```

```

        tip = Polygon(
            np.array([0, tip_length / 2,
                    # 尖端顶点
                    np.array([-tip_width / 2, -
tip_length / 2, 0]), # 左下
                    np.array([tip_width / 2, -tip_length
/ 2, 0]), # 右下
                    color=color,
                    fill_color=color,
                    fill_opacity=1,
                    stroke_width=0, # 无描边，纯
填充
                )
            # 5. 旋转使尖端指向 unit_dir（默认
尖端朝上，即 (0,1,0)）
            angle = np.arctan2(unit_dir[1],
unit_dir[0]) - np.pi / 2
            tip.rotate(angle,
                about_point=ORIGIN)
            # 6. 将箭头放置到中点
            tip.move_to(mid)
            # 7. 组合返回
            return VGroup(line, tip)

class GratingEquationDerive(Scene):

```

合，只返回点或空箭头 <pre> return VGroup(line) unit_dir = direction / norm # 4. 创建等腰三角形箭头 </pre>	<pre> def construct(self): # 全局设置 self.camera.background_color = WHITE CN_FONT = "SimHei" # 若无此 字体，可改为 "Arial" 或删除 font 参数 </pre>
<pre> N=4#缝数 d = 1.5#缝栅间距 b=0.1#缝宽度 theta = np.pi / 12#入射角度 incident_length = 2.6 # 入射光长 ls=[] # 2. 绘制光栅（多缝等距） for i in range(N+1): ls.append(i * d) grating = VGroup() for i in ls: slit = Line(start= i * DOWN, end= (i+d-b)* DOWN, color=BLACK, stroke_width=8) grating.add(slit) grating.move_to(ORIGIN + LEFT * 5) </pre>	<pre> p_line=Line(start=UP*4, end=DOWN*4, color=BLACK, stroke_width=8) p_line.shift(RIGHT*3) self.play(FadeIn(p_line,grating,tuto u_image),run_time=1.5) self.wait(1) # 3. 标注光栅常数 d d_line=DoubleArrow(LEFT * 5+d /2 * UP +LEFT*0.25,LEFT * 5+d /2 * DOWN +LEFT*0.25,color=BLACK,stroke_width=2, buff=0) d_line_U=Line(start=d_line.get_start()+LEFT*0 .25, end=d_line.get_start()+RIGHT* 0.25, color=BLACK, stroke_width=2, buff=0) </pre>

<pre> #显示凸透镜 tutou_image=SVGMOBJect(r"asset s/svg_images/thin_convex_lens.svg") tutou_image.scale(3.5).shift(LEFT* 2.5) tutou_image.set_z_index(100) #tutou_image=DoubleArrow(</pre>	<pre> d_line_D=Line(start=d_line.get_end()+LEFT*0. 25, end=d_line.get_end()+RIGHT* 0.25, color=BLACK, stroke_width=2, buff=0) </pre>
<pre> d_label = MathTex(r"d",color=BLACK, font_size=30).next_to(d_line, LEFT, buff=0.2) d_mark=VGroup(d_line_U,d_line_ D,d_label,d_line) self.play(Write(d_mark)) self.wait(1) source=[] for i in range(N): source.append(grating[i].get_e nd()) play_multi_wavefront(self, source_points=source, max_radius=1.5, speed=1.0, period=0.8, num_waves=3, colors=[BLACK,BLACK,BLACK, BLACK], show_source=False, </pre>	<pre> start=grating[i].get_end(), end=grating[i].get_end()+RIGHT*incident _length*np.cos(theta)+UP*incident_lengt h*np.sin(theta), color=BLACK, stroke_width=4) incident_rays.add(ray) self.play(Create(incident_rays), run_time=1,lag_ratio=0) self.wait(1) # 5. 衍射角 θ h_line_group = VGroup() for i in range(N-1): help_line=Line(start=incident_rays[i][0].get _end(), end=incident_rays[i][0].get _end(), color=BLACK, stroke_width=4) </pre>

<pre> show_direction=False,) # 4. 入射平行光 incident_rays = VGroup() for i in range(N): ray = MidpointArrow(</pre>	<pre> h_line=DashedLine(start=incident_rays[i][0].get _start(), end=incident_rays[i][0].get_start()+DOWN N*(d-0.05)*np.cos(theta)+RIGHT*(d- 0.05)*np.sin(theta), color=BLACK, stroke_width=4) h_line_group.add(h_line) </pre>
<pre> h_line_pin=DashedLine(start=incident_rays[2][0].get_st art(), end=incident_rays[2][0].get_st art()+RIGHT*d, color=BLACK, stroke_width=4) self.play(FadeIn(h_line_group,h_line_ pin),lag_ratio=0.0) theta_label1 = MathTex(r"\theta", font_size=30,color=BLACK).next_to(h_line _group[0], DOWN, buff=0.2) theta_angle=Angle(grating[3],h_li ne_group[1], quadrant=(1,1), radius=0.4, color=BLACK, stroke_width=3) right_angle=RightAngle(incident_r ays[2][0],h_line_group[1], quadrant=(-1,- 1), length=0.15, color=BLACK, stroke_width=2) #theta_label2=theta_label1.copy(). next_to(h_line_pin, UP, buff=0.1).shift(RIGHT*0.5) </pre>	<pre> out_rays = VGroup() for i in range(N): ray = MidpointArrow(start=incident_rays[i][0].get _end()+RIGHT*0.1, end=point, color=BLACK, stroke_width=4) out_rays.add(ray) ray=MidpointArrow(start=incident_rays[0][0].get_e nd(), end=point, color=BLACK, stroke_width=4) out_rays[0]=ray self.play(Create(out_rays), run_time=1,lag_ratio=0) </pre>

<pre> self.play(Write(theta_label1),Write(theta_angle),Write(right_angle)) self.wait(1) point=np.array([3,3,0]) </pre>	<pre> P_dot=Dot(point,color=RED,stroke_width=6) P_dot_label = MathTex(r"P", font_size=30,color=RED).next_to(P_dot, RIGHT, buff=0.2) self.play(Write(P_dot),Write(P_dot_label)) self.wait(1) </pre>
<pre> def play_wavefront(scene, source_point=LEFT * 4, # 波源位置 max_radius=10, # 最大扩散半径（到达后消失） speed=2.0, # 波速 period=0.8, # 波前发射间隔（秒） num_waves=10, # 波前数量 color=BLUE, # 波前颜色 stroke_width=3, # 波前线宽 start_angle=-PI / 2, # 圆弧起始角度（右侧半圆） angle=PI, # 圆弧覆盖角度 show_source=True, # 是否显示波源标记 show_direction=True, # 是否显示传播方向箭头 </pre>	<pre> if radius <= 0 or radius > max_radius: return VGroup() # 透明度随半径增大而降低，到达 max_radius 时完全透明（消失） alpha = 1 - radius / max_radius arc = Arc(radius=radius, start_angle=start_angle, angle=angle, arc_center=source_point, stroke_color=color, stroke_width=stroke_width,) arc.set_stroke(opacity=alpha) return arc # 时间跟踪器 time_tracker = ValueTracker(0) # 动画总时长：最后一个波前发射后传播到最大半径 total_time = (num_waves - 1) * period + max_radius / speed </pre>

<pre>): # 内部辅助函数：根据当前时间计算 第 index 个波前的圆弧 def get_wavefront_arc(index, current_time): emit_time = index * period radius = speed * (current_time - emit_time) # 尚未发出或已超出最大半径： 返回空对象（不可见） </pre>	<pre> if show_source: source_dot = Dot(source_point, color=YELLOW) source_label = Text("波源", font_size=24).next_to(source_dot, DOWN) scene.add(source_dot, source_label) </pre>
<pre> # 方向指示 if show_direction: direction_arrow = Arrow(source_point, source_point + RIGHT * 2, buff=0, color=WHITE,) direction_label = Text("传播方向", font_size=24).next_to(direction_arrow, UP) scene.add(direction_arrow, direction_label) # 创建所有波前圆弧（使用 always_redraw 实时更新） wavefronts = VGroup() for i in range(num_waves): wf = always_redraw(lambda i=i: get_wavefront_arc(i, </pre>	<pre> run_time=total_time, rate_func=linear,) def play_multi_wavefront(scene, source_points, # 波源位 置列表，例如 [LEFT*4, ORIGIN, RIGHT*3] max_radius=10, # 最大 扩散半径（所有源统一） speed=2.0, # 波速 （所有源统一） period=0.8, # 波前发 射间隔（所有源统一） num_waves=10, # 每个 源的波前数量（统一） colors=None, # 可 选：每个源的颜色列表，长度需与 source_points 一致 stroke_width=3, # 波前 线宽 </pre>

<pre> time_tracker.get_value())) wavefronts.add(wf) scene.add(wavefronts) # 播放动画：时间线性推进 scene.play(time_tracker.animate.set_value(total_time), </pre>	<pre> start_angle=-PI / 2, # 圆弧起始角度（统一，可后续扩展为列表） angle=PI, # 圆弧覆盖角度（统一） show_source=True, # 是否显示波源标记 show_direction=False, # 是否显示传播方向（多源时通常关闭，以免箭头过多）): </pre>
<pre> if colors is None: colors = [BLUE] * len(source_points) else: assert len(colors) == len(source_points), "colors 列表长度必须与 source_points 相同" # 时间跟踪器（所有源共享） time_tracker = ValueTracker(0) total_time = (num_waves - 1) * period + max_radius / speed # 为每个波源添加标记（可选） if show_source: for pt in source_points: dot = Dot(pt, color=YELLOW) label = Text("波源", font_size=24).next_to(dot, DOWN) </pre>	<pre> for idx, (src, col) in enumerate(zip(source_points, colors)): # 内部辅助函数：返回第 wave_index 个波前在当前时间下的圆弧 def get_arc(src=src, col=col, wave_index=idx, current_time=None): # 注意：这里 wave_index 参数被外层 idx 覆盖，实际需要为每个 wave 单独生成闭包 pass # 占位，实际在下方循环中创建 # 创建该源的 num_waves 个波前 for wave_idx in range(num_waves): # 使用闭包捕获 src, col, wave_idx def make_wavefront(src=src, col=col, wave_idx=wave_idx): def updater(): </pre>

<pre> scene.add(dot, label) # 为每个波源添加方向指示 (可选) if show_direction: for pt in source_points: arrow = Arrow(pt, pt + RIGHT * 2, buff=0, color=WHITE) scene.add(arrow) # 为每个波源创建波前圆弧组 all_wavefronts = VGroup() </pre>	<pre> current_time = time_tracker.get_value() emit_time = wave_idx * period radius = speed * (current_time - emit_time) if radius <= 0 or radius > max_radius: return VGroup() alpha = 1 - radius / max_radius arc = Arc(</pre>
<pre> radius=radius, start_angle=start_a ngle, angle=angle, arc_center=src, stroke_color=col, stroke_width=strok e_width,) arc.set_stroke(opacity= alpha) return arc return always_redraw(updater) wf = make_wavefront(src, col, wave_idx) all_wavefronts.add(wf) scene.add(all_wavefronts) </pre>	

```
# 一次性播放整个时间推进动画

scene.play(

    time_tracker.animate.set_value(total_time),

    run_time=total_time,

    rate_func=linear,

)
```

#single_slit.py

<pre>from manim import * import numpy as np class SingleSlitDiffraction(Scene): def construct(self): self.camera.background_color = WHITE # 参数设置 x_min, x_max = -3, 3 # 创建坐标系 axes = Axes(x_range=[x_min, x_max, 1], # 步长改为 1, 对应λ/a 的整数倍 y_range=[0, 1.2, 0.2], axis_config={"color": BLACK, "stroke_width": 2}, x_axis_config={ "numbers_to_include":</pre>	<pre># x 轴标签: λ/a x_label = MathTex(r"\lambda / a", color=BLACK).scale(0.8) _label.next_to(axes.x_axis.get_right(), DOWN, buff=0.2) # y 轴标签: I/I_0 y_label = MathTex(r"I / I_0", color=BLACK).scale(0.8) y_label.next_to(axes.y_axis.get_top(), RIGHT, buff=0.2) self.play(Create(axes), Write(x_label), Write(y_label)) # 强度曲线 intensity = lambda x: np.sinc(x) ** 2 graph = axes.plot(intensity, x_range=[x_min, x_max], color="#1B088A", stroke_width=2.5,</pre>
---	--

<pre> np.arange(x_min, x_max + 1, 1), "color": BLACK, }, y_axis_config={ "numbers_to_include": np.arange(0, 1.4, 0.2), "color": BLACK, }, tips=False,) </pre>	<pre>) shadow=axes.get_area(graph=graph, color=BLUE, opacity=0.3,) self.play(Create(graph), run_time=4) self.play(FadeIn(shadow)) self.wait(2) </pre>
<pre> # 在 x 轴下方标注关键位置数字 # 暗纹位置: $x = \pm 1, \pm 2$ # 明纹位置: $x \approx \pm 1.43, \pm 2.46$ 2.46: "2.46", } x_ticks = VGroup() for x_val, text in x_positions.items(): tick = Text(text, color=BLACK).scale(0.35) tick.next_to(axes.c2p(x_val, 0), DOWN, buff=0.2) x_ticks.add(tick) self.play(Write(x_ticks), run_time=2) # 标记明纹位置 x_positions = { -2.46: "-2.46", -2.0: "-2", </pre>	<pre> (2.4590, 0.0165, "0.0165") ', (- 2.4590, 0.0165, "0.0165"),] for x_val, i_val, i_text in bright_fringes: # 红点标记明纹位置 point = Dot(axes.c2p(x_val, i_val), color=RED, radius=0.08) # I/I_0 数值标注在点上方 label = MathTex(r"I/I_0 = " + i_text, color=RED).scale(0.5) label.next_to(point, UP, buff=0.15) self.play(FadeIn(point), Write(label), </pre>

```
-1.43: "-1.43",
-1.0: "-1",
0: "0",
1.0: "1",
1.43: "1.43",
2.0: "2",
bright_fringes = [
    (0, 1.0, "1.0000"),
    (1.4303, 0.0472, "0.0472")
],
(-
1.4303, 0.0472, "0.0472"),
```

```
run_time=0.6
)
self.wait(3)
```

(2) 绘图代码

generate_figures.py
generate_fringe_patterns.py
generate_resolving_power.py

#generate_figures.py

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
# 中文字体
matplotlib.rcParams['font.sans-serif'] =
['Microsoft YaHei', 'SimHei', 'Noto Sans
SC', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'
] = False
# 物理参数 —— 窄包络配置, 让调制
结构清晰可见
A = 3.0 # 缝宽 a (增大
```

```
with np.errstate(divide='ignore',
invalid='ignore'):
    l_single = np.where(
        np.abs(alpha) < 1e-12, 1.0,
        (np.sin(alpha) / alpha) ** 2
    )
    l_multi = np.where(
        np.abs(np.sin(beta)) < 1e-12,
        np.where(np.abs(beta % np.pi) <
1e-12, float(N ** 2), 0.0),
        (np.sin(N * beta) / np.sin(beta))
** 2
```

→ 包络更窄, 结构更丰富) <pre> D = 12.0 # 光栅常数 d (d/a = 4, 每第 4 级干涉峰缺级) N = 6 # 缝数 WAVELENGTH = 1.0 # 波长 λ # 数据计算 SIN_THETA = np.linspace(-1.0, 1.0, 4000) alpha = np.pi * A * SIN_THETA / WAVELENGTH beta = np.pi * D * SIN_THETA / WAVELENGTH ax.set_xlim(-1.0, 1.0) ax.set_ylim(-0.05 * y_max, y_max * 1.12) ax.set_xlabel('sin θ', fontsize=18, fontweight='bold', color='black', labelpad=8) ax.set_ylabel('相\n对\n强\n度\n', fontsize=18, fontweight='bold', color='black', rotation=0, labelpad=8, va='center') ax.tick_params(labelsize=13, colors='black') ax.set_xticklabels([]) ax.set_yticklabels([]) ax.set_facecolor('white') for spine in ax.spines.values(): spine.set_color('black') def save_figure(fig, filename): fig.savefig(filename, dpi=200, facecolor=fig.get_facecolor(), </pre>	<pre>) I_total = I_single * I_multi # 共享配置 FIG_W, FIG_H = 12.8, 7.2 # 16:9 COLOR_SINGLE = '#3b82b6' COLOR_MULTI = '#ef4444' COLOR_TOTAL = '#8b5cf6' def style_ax(ax, y_max=1.15): """统一轴样式 —— 白底, 无网 格, 无标题, 粗体黑色坐标""" ax1.plot(SIN_THETA, I_single, color=COLO R_SINGLE, linewidth=2.8, zorder=3) save_figure(fig1, '01_single_slit.png') plt.close(fig1) # 图 2: 多缝干涉因子 —— 纯曲线 fig2, ax2 = plt.subplots(figsize=(FIG_W, FIG_H), facecolor='white') style_ax(ax2, y_max=N**2) ax2.plot(SIN_THETA, I_multi, color=COLOR_MULTI, linewidth=1.3, zorder=3) save_figure(fig2, '02_multi_slit.png') plt.close(fig2) # 图 3: 光栅总光强 —— 调制叠加 (保 留包络虚线以体现调制关系) fig3, ax3 = plt.subplots(figsize=(FIG_W, FIG_H), facecolor='white') style_ax(ax3, y_max=N**2) ax3.fill_between(SIN_THETA, 0, I_total, color=COLOR_TOTAL, alpha=0.22, </pre>
---	--

```

        edgecolor='none',
bbox_inches='tight', pad_inches=0.3)

    print(f"  -> 已保存: {filename}")
# 图 1: 单缝衍射因子 —— 纯曲线
fig1, ax1 = plt.subplots(figsize=(FIG_W,
FIG_H), facecolor='white')

style_ax(ax1, y_max=1.0)

ax1.fill_between(SIN_THETA, 0, I_single,
color=COLOR_SINGLE, alpha=0.18,
linewidth=0)

linewidth=0)

ax3.plot(SIN_THETA, I_total,
color=COLOR_TOTAL, linewidth=2.2,
zorder=4

ax3.plot(SIN_THETA, I_single * N**2,
color=COLOR_SINGLE, linewidth=1.8,

linestyle='--', alpha=0.5,
zorder=3)

save_figure(fig3, '03_grating_total.png')

plt.close(fig3)

print("\n 三张图片全部生成完毕! ")

```

#generate_fringe_patterns.py

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib

from matplotlib.colors import
LinearSegmentedColormap

# 中文字体
matplotlib.rcParams['font.sans-serif'] =
['Microsoft YaHei', 'SimHei', 'Noto Sans
SC', 'DejaVu Sans']

matplotlib.rcParams['axes.unicode_minus'
] = False

# 物理参数 —— 与
generate_figures.py 完全一致

A = 3.0          # 缝宽 a (增大
→ 包络更窄, 结构更丰富)

    I_single = np.where(
        np.abs(alpha) < 1e-12, 1.0,
        (np.sin(alpha) / alpha) ** 2
    )

    I_multi = np.where(
        D = 12.0          # 光栅常数 d
        (d/a = 4, 每第 4 级干涉峰缺级)

        N = 6          # 缝数

        WAVELENGTH = 1.0    # 波长 λ

        # 数据计算

        SIN_THETA = np.linspace(-1.0, 1.0, 4000)

        alpha = np.pi * A * SIN_THETA /
        WAVELENGTH

        beta = np.pi * D * SIN_THETA /
        WAVELENGTH

        with np.errstate(divide='ignore',
        invalid='ignore'):

            'green': [(0.0, 1.0, 1.0), (1.0, g, g)],

            'blue': [(0.0, 1.0, 1.0), (1.0, b,
            b)],

        }

        return
        LinearSegmentedColormap(f'white_to_{h

```

```

        np.abs(np.sin(beta)) < 1e-12,
        np.where(np.abs(beta % np.pi) <
1e-12, float(N ** 2), 0.0),
        (np.sin(N * beta) / np.sin(beta))
** 2
    )

I_total = I_single * I_multi
# 共享配置
FIG_W, FIG_H = 12.8, 7.2  # 16:9
COLOR_SINGLE = '#3b82b6'  # 蓝色
—— 单缝衍射
COLOR_MULTI = '#ef4444'  # 红色
—— 多缝干涉
COLOR_TOTAL = '#8b5cf6'  # 紫色
—— 光栅总效果

N_VERTICAL = 600  # 条纹垂
直方向的像素数
Y_TOP = 3.0  # 纵向范
围（任意单位，仅用于显示）
def make_colormap(hex_color):
    """创建从白色(强度=0)到目标颜色
(强度=1)的
LinearSegmentedColormap"""
    r, g, b =
tuple(int(hex_color.lstrip('#')[i:i+2], 16) /
255.0 for i in (0, 2, 4))
    cdict = {
        'red':  [(0.0, 1.0, 1.0), (1.0, r, r)],
def make_fringe_2d(intensity_1d,
gamma=1.0):
    """将一维光强分布扩展为二维条纹
图，gamma < 1 可提升暗纹可见度"""
    i_norm = intensity_1d /
np.max(intensity_1d)
    i_gamma = i_norm ** gamma
# gamma 校正：压缩动态范围，次峰
更可见
    return np.tile(i_gamma,
ex_color.lstrip("#")}', cdict)
def style_fringe_ax(ax):
    """去除所有坐标轴，白底"""
    ax.set_xlim(-1.0, 1.0)
    ax.set_ylim(-Y_TOP, Y_TOP)
    ax.set_facecolor('white')
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_visible(False)
def style_fringe_ax(ax):
    """去除所有坐标轴，白底"""
    ax.set_xlim(-1.0, 1.0)
    ax.set_ylim(-Y_TOP, Y_TOP)
    ax.set_facecolor('white')
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():
        spine.set_visible(False)
def make_fringe_2d(intensity_1d,
gamma=1.0):
    """将一维光强分布扩展为二维条纹
图，gamma < 1 可提升暗纹可见度"""
    i_norm = intensity_1d /
np.max(intensity_1d)
    i_gamma = i_norm ** gamma
# gamma 校正：压缩动态范围，次峰
更可见
    return np.tile(i_gamma,

```

```

np.max(intensity_1d)

    i_gamma = i_norm ** gamma
# gamma 校正: 压缩动态范围, 次峰更
可见

    return np.tile(i_gamma,
(N_VERTICAL, 1))

def save_figure(fig, filename):

    fig.savefig(filename, dpi=200,
facecolor=fig.get_facecolor(),

                    edgecolor='none',
bbox_inches='tight', pad_inches=0.3)
    print(f"  -> 已保存: {filename}")

fringe_single = make_fringe_2d(l_single,
gamma=0.35)  # 大幅提升次级峰可见
度

cmap_single =
make_colormap(COLOR_SINGLE)

ax1.imshow(fringe_single,
cmap=cmap_single, aspect='auto',

            extent=[-1.0, 1.0, -Y_TOP,
Y_TOP], origin='lower')

save_figure(fig1,
'01_single_slit_fringe.png')

plt.close(fig1)

# 图 2: 多缝干涉条纹

fig2, ax2 = plt.subplots(figsize=(FIG_W,
FIG_H), facecolor='white')

style_fringe_ax(ax2)

fringe_multi = make_fringe_2d(l_multi)
# 各级主极大等亮, 无需 gamma

cmap_multi =
make_colormap(COLOR_MULTI)

(N_VERTICAL, 1))

def save_figure(fig, filename):

    fig.savefig(filename, dpi=200,
facecolor=fig.get_facecolor(),
edgecolor='none', bbox_inches='tight',
pad_inches=0.3)

    print(f"  -> 已保存: {filename}")

# 图 1: 单缝衍射条纹

fig1, ax1 = plt.subplots(figsize=(FIG_W,
FIG_H), facecolor='white')

style_fringe_ax(ax1)

cmap_total =
make_colormap(COLOR_TOTAL)

ax3.imshow(fringe_total,
cmap=cmap_total, aspect='auto',

            extent=[-1.0, 1.0, -Y_TOP,
Y_TOP], origin='lower')

save_figure(fig3, '03_grating_total_fringe.p
ng')

plt.close(fig3)

print("\n 三张条纹图全部生成完毕! ")

```

```
ax2.imshow(fringe_multi,
cmap=cmap_multi, aspect='auto',
            extent=[-1.0, 1.0, -Y_TOP,
Y_TOP], origin='lower')

save_figure(fig2,
'02_multi_slit_fringe.png')

plt.close(fig2)

fig3, ax3 = plt.subplots(figsize=(FIG_W,
FIG_H), facecolor='white')

style_fringe_ax(ax3)

fringe_total = make_fringe_2d(l_total,
gamma=0.45) # 让包络调制后的外层峰
可见
```

#generate_resolving_power.py

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib

# 中文字体
matplotlib.rcParams['font.sans-serif'] =
['Microsoft YaHei', 'SimHei', 'Noto Sans
SC', 'DejaVu Sans']

matplotlib.rcParams['axes.unicode_minus'
] = False

# 物理参数 (自拟)
K_LIST = [1, 2, 3, 5, 8] # 图 1:
衍射级次
DA_LIST = [3, 4, 6, 8, 12] # 图
2/3: 光栅常数与缝宽之比 d/a
LAMBDA = 589.3 #
图 3 的参考波长 (nm)
# 配色 (经 dataviz 校验) 与共享样式
FIG_W, FIG_H = 12.8, 7.2 # 16:9
INK = '#0b0b0b' #
```

```
图 2/3: 顺序蓝渐变 COLORS_DA =
['#6da7ec', '#3987e5', '#256abf',
'#184f95', '#0d366b']

def style_ax(ax, xlabel, ylabel, xlim, ylim,
logx=False, logy=False):

    """定量对比曲线轴样式 —— 白
底, 粗体黑坐标, 浅色横向发丝网格"""

    if logx:
        ax.set_xscale('log')

    if logy:
        ax.set_yscale('log')

    ax.set_xlim(*xlim)
    ax.set_ylim(*ylim)

    ax.set_facecolor('white')

    ax.set_xlabel(xlabel, fontsize=18,
fontweight='bold', color=INK,
labelpad=8)

    ax.set_ylabel(ylabel, fontsize=18,
fontweight='bold', color=INK,
```

主墨色 (坐标)	labelpad=8)
GRID = '#e1e0d9' #	ax.tick_params(labelsize=13,
发丝网格线	colors=INK, width=1.2)
# 图 1: 类别色 (固定顺序, k 的身份标识)	for spine in ax.spines.values():
	spine.set_color(INK)
COLORS_ORDER = ['#2a78d6',	ax.grid(axis='y', color=GRID,
'#eb6834', '#1baf7a', '#eda100',	linewidth=1.0)
'#e87ba4']	ax.set_axisbelow(True)
def add_legend(ax, loc):	FIG_H), facecolor='white')
ax.legend(loc=loc, frameon=False,	N_MAX2 = 500
fontsize=12.5)	N2 = np.linspace(0, N_MAX2, 800)
def save_figure(fig, filename):	for da, c in zip(DA_LIST, COLORS_DA):
fig.savefig(filename, dpi=200,	k_max = da - 1
facecolor=fig.get_facecolor(),	ax2.plot(N2, k_max * N2, color=c,
edgecolor='none',	linewidth=3.0,
bbox_inches='tight', pad_inches=0.3)	label=f'd/a = {da} (最高
print(f" -> 已保存: {filename}")	级次 k = {k_max}) ', zorder=3)
# 图 1: R = kN 随缝数 N 的变化 (不同级次 k)	style_ax(ax2, '缝数 N', '色分辨本领 R =
fig1, ax1 = plt.subplots(figsize=(FIG_W,	$\lambda/\Delta\lambda$ ', (0, N_MAX2 * 1.12), (0, 11 * N_MAX2 * 1.06))
FIG_H), facecolor='white')	add_legend(ax2, 'upper left')
N_MAX1 = 400	save_figure(fig2, '考虑缺级: 最高可用级
N1 = np.linspace(0, N_MAX1, 800)	次下的色分辨本领.png')
	plt.close(fig2)
for k, c in zip(K_LIST, COLORS_ORDER):	# 图 3: 可分辨最小波长差 $\Delta\lambda = \lambda/R$ 随
ax1.plot(N1, k * N1, color=c,	N 的变化 ($\lambda = 589.3 \text{ nm}$)
linewidth=3.0, label=f'k = {k}', zorder=3)	fig3, ax3 = plt.subplots(figsize=(FIG_W,
ax1.text(N_MAX1 * 1.015, k * N_MAX1,	FIG_H), facecolor='white')
f'k={k}', color=c, fontsize=13, fontweight='	N3 = np.logspace(0, 4, 800)
bold', va='center')	for da, c in zip(DA_LIST, COLORS_DA):
style_ax(ax1, '缝数 N', '色分辨本领 R =	k_max = da - 1
$\lambda/\Delta\lambda$ ', (0, N_MAX1 * 1.12), (0, 8 *	

<pre> N_MAX1 * 1.06)) add_legend(ax1, 'upper left') save_figure(fig1, '光栅色分辨本领随缝数的变化.png') plt.close(fig1) # 图 2: 考虑缺级 —— 最高可用级次下的 R 随 N 的变化 (不同 d/a) fig2, ax2 = plt.subplots(figsize=(FIG_W, style_ax(ax3, '缝数 N', '可分辨的最小波长差 $\Delta\lambda$ / nm', (1, 10**4 * 1.18), (0.001, 320), logx=True, logy=True) add_legend(ax3, 'upper right') </pre>	<pre> ax3.plot(N3, LAMBDA / (k_max * N3), color=c, linewidth=3.0, label=f'd/a = {da} (最高 级次 k = {k_max}) ', zorder=3) ax3.text(10**4 * 1.02, LAMBDA / (k_max * 10**4), f'{da}', color=c, fontsize=13, fontweight='bold', va='center') save_figure(fig3, '可分辨的最小波长差随 缝数的变化.png') plt.close(fig3) print("\n 三张色分辨本领对比曲线全部生 成完毕! ") </pre>
---	--